ICS 202 – Data Structures and Algorithms

Topic 09: Heaps

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Notation. A heap is written as its array, from index 1 (the root) to index n, as in the slides: the parent of index i is $\lfloor i/2 \rfloor$ and its children are $2i$ and $2i+1$. In an array that describes a tree, "–" means an empty position (no node). Percolate down moves the item to the smaller child (min-heap) or the larger child (max-heap), using the left child when the two children are equal.

Part A: Multiple choice

Q1.

Unit 09 Activity 01 (T252)

Each of the following arrays is the level-order (array) representation of a complete binary tree:

- Tree A: 12, 20, 15, 23, 29, 19, 48, 60, 26, 30
- Tree B: 12, 20, 21, 6, 29, 19, 48, 60, 26, 30
- Tree C: 13, 21, 16, 24, 31, 19, 68, 65, 26, 32
- Tree D: 70, 67, 50, 34, 40, 33, 25, 20, 29, 35

Which classification is correct?

(A) A: MinHeap; B: not a heap; C: MinHeap; D: MaxHeap

(B) A: MinHeap; B: MinHeap; C: MinHeap; D: MaxHeap

(C) A: MinHeap; B: not a heap; C: not a heap; D: MaxHeap

(D) A: not a heap; B: not a heap; C: MinHeap; D: MaxHeap

Answer: (A). In B, key 6 (index 4) is smaller than its parent 20 (index 2), so B is neither; A and C satisfy the MinHeap property and D the MaxHeap property.

My answer

Q2.

Quiz 3 Ver A (T241)

Which of the following is a valid max heap?

(A) 32, 31, 26, 24, 21, 19, –, 12, 15, 13

(B) 32, 31, 26, 24, 21, 19, 17, 12, 15, –, 13

(C) 32, 31, 26, 24, 21, 19, 17, 12, 15, 13

(D) 12, 20, 26, 24, 21, 41, 37, 42, 35, 33

Answer: (C). (A) and (B) have a gap, so they are not complete trees; (D) has children larger than their parents.

My answer

Q3. Quiz 4 Ver A (T251)

Given the following three trees (in array form):

- Tree 1: 13, 21, 16, 24, 31, 19, –, 65, 26, 32
- Tree 2: 68, 65, 46, 24, 32, 23, 25, 15, 20, 31
- Tree 3: 68, 65, 46, 67, 32, 23, 25, 15, 20, 31

(A) Only Tree 1 is a heap.

(B) Only Tree 2 is a heap.

(C) Both Tree 2 and Tree 3 are heaps.

(D) All trees are not heaps.

Answer: (B). Tree 1 is not complete (16 has no right child but the next level has nodes); in Tree 3, 67 $>$ its parent 65.

My answer

Q4. Final exam (T211)

For a MaxHeap in which indexes start at 1, the index of the parent of the node at index k ($k > 1$) and the index of its right child (if any) are:

(A) parent $\lfloor k/2 \rfloor$; right child $2k + 1$

(B) parent $\lfloor (k-1)/2 \rfloor$; right child $2k + 2$

(C) parent $\lfloor k/2 \rfloor$; right child $2k$

(D) parent $2k$; right child $\lfloor k/2 \rfloor + 1$

Answer: (A). With the root at index 1, parent $= \lfloor k/2 \rfloor$, left $= 2k$, right $= 2k + 1$.

My answer

Q5. Major exam (T222)

A min-heap has root 13. The children of 13 are 21 (left) and 16 (right); the children of 21 are 24 and 31; the children of 16 are 19 and 68; the children of 24 are 65 and 26; 31 has only a left child, 32. What is its array representation (indexes 1 to 10)?

(A) 13, 21, 16, 24, 31, 19, 68, 65, 26, 32

(B) 13, 21, 24, 65, 26, 31, 32, 16, 19, 68

(C) 13, 21, 16, 24, 31, 19, 68, 32, 65, 26

(D) 65, 24, 26, 21, 32, 31, 13, 19, 16, 68

Answer: (A). The keys are stored in breadth-first (level) order, left to right.

My answer

Q6. Unit 09 Activity 01 (T252)

A heap has root 12 with children 20 (left) and 15 (right); 20 has children 23 and 29; 15 has children 19 and 48; 23 has children 60 and 26; 29 has only a left child, 30. In its array representation (index 0 unused), at which index is 30 stored, and which key is at index 6?

(A) index 10; key 19

(B) index 9; key 48

(C) index 10; key 48

(D) index 11; key 19

Answer: (A). The array is 12, 20, 15, 23, 29, 19, 48, 60, 26, 30, so 30 is at index 10 and index 6 holds 19.

My answer

Q7. Unit 09 Activity 01 (T252)

A heap is stored in an array (index 0 unused) as: 23, 31, 26, 34, 41, 29, 78, 75, 36, 42 (indexes 1 to 10). Which statement about the drawn tree is true?

(A) The children of 34 are 75 and 36, and 42 is the left child of 41.

(B) The children of 31 are 41 and 29.

(C) 42 is the right child of 36.

(D) The parent of 29 is 31.

Answer: (A). 34 is at index 4 (children at 8 and 9: 75, 36); 41 is at index 5 (left child at 10: 42).

My answer

Q8. Major exam (T222)

While building a max-heap we get the heap 10, 4, 6. What is the heap after inserting 20?

(A) 20, 10, 6, 4

(B) 20, 4, 6, 10

(C) 10, 4, 6, 20

(D) 20, 6, 10, 4

Answer: (A). 20 goes to index 4, swaps with its parent 4, then with the root 10.

My answer

Q9.

Unit 09 Activity 01 (T252)

Insert 15 into the min-heap 13, 18, 16, 24, 21, 19, 68, 65, 26, 32, 31, performing the required percolation. The resulting heap is:

(A) 13, 18, 15, 24, 21, 16, 68, 65, 26, 32, 31, 19

(B) 13, 15, 16, 24, 18, 19, 68, 65, 26, 32, 31, 21

(C) 13, 18, 16, 24, 21, 15, 68, 65, 26, 32, 31, 19

(D) 13, 18, 16, 24, 21, 19, 68, 65, 26, 32, 31, 15

Answer: (A). 15 is placed at index 12 (child of 19), swaps with 19, then with 16, and stops below 13.

My answer

Q10.

Final exam (T131)

The values 5, 12, 20, 10, 15, 30, 26, 23, 25 are inserted, in this order, into an initially empty MinHeap. The resulting heap is:

(A) 5, 10, 20, 12, 15, 30, 26, 23, 25

(B) 5, 12, 20, 10, 15, 30, 26, 23, 25

(C) 5, 10, 20, 12, 15, 26, 30, 23, 25

(D) 5, 10, 15, 12, 20, 30, 26, 23, 25

Answer: (A). Only 10 percolates up (swaps with 12); every other key is already larger than its parent.

My answer

Q11.

Final exam (T161)

Consider the MinHeap 3, 10, 5, 11, 12, 6, 8, 15, 20, 13. What is the heap after Enqueue(5), Enqueue(1), Enqueue(2), in this order?

(A) 1, 5, 2, 11, 10, 3, 8, 15, 20, 13, 12, 6, 5

(B) 1, 2, 3, 11, 10, 5, 8, 15, 20, 13, 12, 6, 5

(C) 1, 5, 2, 11, 10, 3, 8, 15, 20, 13, 12, 5, 6

(D) 1, 10, 2, 11, 5, 3, 8, 15, 20, 13, 12, 6, 5

Answer: (A). After Enqueue(5): 3, 5, 5, 11, 10, 6, 8, 15, 20, 13, 12. 1 then percolates from index 12 to the root, and 2 from index 13 up to index 3.

My answer

Q12. Quiz 3 Ver A (T242)

Deleting the minimum value in a min-heap costs

(A) $\Theta(1)$

(B) $O(\log n)$

(C) $\Theta(\sqrt{n})$

(D) $\Theta(n)$

Answer: (B). The last key replaces the root and percolates down at most the height of the tree, $O(\log n)$.

My answer

Q13. Quiz 4 Ver A (T251)

Deleting the maximum value in a min-heap costs

(A) $O(n)$

(B) $\Omega(n \log n)$

(C) $O(\log n)$

(D) $O(\sqrt{n})$

Answer: (A). The maximum can be any leaf, and about $n/2$ keys are leaves, so finding it takes linear time.

My answer

Q14. Unit 09 Activity 01 (T252)

Delete 18 (the minimum) from the min-heap 18, 21, 19, 24, 31, 23, 68, 65, 26, 32, performing percolation if necessary. The result is:

(A) 19, 21, 23, 24, 31, 32, 68, 65, 26

(B) 21, 24, 19, 26, 31, 23, 68, 65, 32

(C) 19, 21, 32, 24, 31, 23, 68, 65, 26

(D) 32, 21, 19, 24, 31, 23, 68, 65, 26

Answer: (A). 32 replaces the root and percolates down: swap with 19 (smaller child), then with 23.

My answer

Q15. Final exam (T211)

Give the heap, as an array, after deleting the maximum from the max-heap 20, 18, 14, 12, 11, 13, 7, 6, 8, 5, 9, 4.

(A) 18, 12, 14, 8, 11, 13, 7, 6, 4, 5, 9

(B) 18, 12, 14, 6, 11, 13, 7, 4, 8, 5, 9

(C) 4, 18, 14, 12, 11, 13, 7, 6, 8, 5, 9

(D) 18, 11, 14, 12, 4, 13, 7, 6, 8, 5, 9

Answer: (A). 4 replaces 20 and percolates down through the larger child each time: 18, then 12, then 8.

My answer

Q16. Final exam (T161)

Starting from the MinHeap 1, 5, 2, 11, 10, 3, 8, 15, 20, 13, 12, 6, 5, three Dequeue() operations are carried out. The resulting heap is:

(A) 5, 10, 5, 11, 12, 6, 8, 15, 20, 13

(B) 3, 5, 5, 11, 10, 6, 8, 15, 20, 13, 12

(C) 5, 5, 6, 11, 10, 12, 8, 15, 20, 13

(D) 5, 10, 6, 11, 12, 5, 8, 15, 20, 13

Answer: (A). After each deleteMin: 2, 5, 3, . . . ; then 3, 5, 5, 11, 10, 6, 8, 15, 20, 13, 12; then 5, 10, 5, 11, 12, 6, 8, 15, 20, 13 (equal children: the left one is chosen).

My answer

Q17. Final exam (T131)

Consider the MinHeap 5, 10, 20, 12, 15, 30, 26, 23, 25. What is the MinHeap after deleting the root node?

(A) 10, 12, 20, 23, 15, 30, 26, 25

(B) 10, 15, 20, 12, 25, 30, 26, 23

(C) 25, 10, 20, 12, 15, 30, 26, 23

(D) 10, 12, 20, 25, 15, 30, 26, 23

Answer: (A). 25 moves to the root and percolates down: swap with 10, then 12, then 23.

My answer

Q18. Quiz 3 Ver A (T241)

In deleting an arbitrary key from a min-heap, you may need to

(A) percolate up only.

(B) percolate down only.

(C) percolate up or percolate down.

(D) percolate up, percolate down, or do no percolation at all.

Answer: (D). The last key copied into the hole can be smaller than the new parent (up), larger than a child (down), or fit exactly (nothing; also when the deleted key is the last node).

My answer

Q19. Unit 09 Activity 01 (T252)

Delete 25 from the min-heap 10, 20, 25, 28, 20, 30, 40, 30, 90, 70, 80, 60, 32, 50, 40, 35 and perform the required percolation. The result is:

(A) 10, 20, 30, 28, 20, 32, 40, 30, 90, 70, 80, 60, 35, 50, 40

(B) 10, 20, 35, 28, 20, 30, 40, 30, 90, 70, 80, 60, 32, 50, 40

(C) 10, 20, 30, 28, 20, 35, 40, 30, 90, 70, 80, 60, 32, 50, 40

(D) 10, 20, 40, 28, 20, 30, 35, 30, 90, 70, 80, 60, 32, 50, 40

Answer: (A). The last key 35 replaces 25 (index 3) and percolates down: swap with 30, then with 32.

My answer

Q20. Final exam (T181)

For the min-heap 13, 21, 16, 24, 31, 19, 68, 65, 26, 32, what is the resulting min-heap after deleting element 21?

(A) 13, 24, 16, 26, 31, 19, 68, 65, 32

(B) 13, 31, 16, 24, 32, 19, 68, 65, 26

(C) 13, 32, 16, 24, 31, 19, 68, 65, 26

(D) 13, 24, 16, 32, 31, 19, 68, 65, 26

Answer: (A). 32 replaces 21 and percolates down: swap with 24, then with 26.

My answer

Q21. Quiz 3 Ver A (T242)

Delete key 31 from the min-heap 8, 9, 11, 31, 10, 11, 16, 46, 51, 32, 21, 13. The result is:

(A) 8, 9, 11, 13, 10, 11, 16, 46, 51, 32, 21

(B) 8, 9, 11, 46, 10, 11, 16, 13, 51, 32, 21

(C) 8, 9, 11, 10, 13, 11, 16, 46, 51, 32, 21

(D) 8, 9, 11, 21, 10, 11, 16, 46, 51, 32, 13

Answer: (A). The last key 13 replaces 31 (index 4); its children 46 and 51 are larger and its parent 9 is smaller, so no percolation is needed.

My answer

Q22. Unit 09 Activity 01 (T252)

In the min-heap 18, 21, 19, 24, 31, 23, 68, 65, 26, 32, change the key 21 to 11 and perform the necessary percolation. The result is:

(A) 11, 18, 19, 24, 31, 23, 68, 65, 26, 32

(B) 18, 11, 19, 24, 31, 23, 68, 65, 26, 32

(C) 11, 19, 18, 24, 31, 23, 68, 65, 26, 32

(D) 11, 21, 19, 24, 31, 23, 68, 65, 26, 32

Answer: (A). 11 $<$ its parent 18, so it percolates up one level.

My answer

Q23. Unit 09 Activity 01 (T252)

In the min-heap 18, 21, 19, 24, 31, 23, 68, 65, 26, 32, change the key 21 to 30 and perform the necessary percolation. The result is:

(A) 18, 24, 19, 26, 31, 23, 68, 65, 30, 32

(B) 18, 30, 19, 24, 31, 23, 68, 65, 26, 32

(C) 18, 24, 19, 30, 31, 23, 68, 65, 26, 32

(D) 18, 31, 19, 24, 30, 23, 68, 65, 26, 32

Answer: (A). 30 percolates down: swap with the smaller child 24, then with 26.

My answer

Q24. Quiz 4 V1 (T251)

Given the min-heap 1, 2, 3, 8, 19, 16, 7, 12, 40, replace 2 with 15. The updated heap is:

(A) 1, 8, 3, 12, 19, 16, 7, 15, 40

(B) 1, 15, 3, 8, 19, 16, 7, 12, 40

(C) 1, 8, 3, 15, 19, 16, 7, 12, 40

(D) 1, 3, 15, 8, 19, 16, 7, 12, 40

Answer: (A). 15 percolates down: swap with 8 (smaller of 8 and 19), then with 12.

My answer

Q25. [new]

A heap is built top-down by inserting n keys, one at a time, into an initially empty heap (each insertion percolates the new key up). The worst-case complexity of building the heap is

(A) $O(\log n)$

(B) $O(n)$

(C) $O(n \log n)$

(D) $O(n^2)$

Answer: (C). Each of the n insertions costs $O(\log n)$.

My answer

Q26. Quiz 3 Ver B (T241)

For the array with keys 40, 39, 38, 36, 26, 25, 23, 20, respectively, the resulting min-heap after applying the top-down algorithm is

(A) 20, 23, 25, 36, 38, 39, 26, 40

(B) 20, 26, 23, 36, 40, 25, 38, 39

(C) 40, 39, 38, 36, 26, 25, 23, 20

(D) 20, 23, 25, 26, 36, 38, 39, 40

Answer: (A). Percolate up each element from index 2 to index 8 (B is the bottom-up result).

My answer

Q27. Unit 09 Activity 02 (T252)

Convert the array 4, 6, 3, 2, 16, 9, 10, 14, 8, 7 into a MaxHeap using the top-down method. The result is:

(A) 16, 14, 10, 8, 7, 3, 9, 2, 6, 4

(B) 16, 14, 10, 8, 7, 9, 3, 2, 4, 6

(C) 16, 14, 10, 9, 8, 7, 6, 4, 3, 2

(D) 16, 14, 9, 8, 7, 3, 10, 2, 6, 4

Answer: (A). Percolate up the elements at indexes 2, 3, . . . , 10 in turn (B is the bottom-up result).

My answer

Q28. Quiz 3 Ver A (T241)

The complexity of converting an array into a min-heap is $\Theta(n)$ in

(A) the top-down algorithm only.

(B) the bottom-up algorithm only.

(C) both the top-down and the bottom-up algorithms.

(D) none of the algorithms.

Answer: (B). Bottom-up heapify is $O(n)$; top-down is $O(n \log n)$.

My answer

Q29. Quiz 3 Ver A (T241)

For the array with keys 40, 39, 38, 36, 26, 25, 23, 20, respectively, the resulting min-heap after applying the bottom-up algorithm is

(A) 20, 23, 25, 36, 38, 39, 26, 40

(B) 20, 26, 23, 36, 40, 25, 38, 39

(C) 40, 39, 38, 36, 26, 25, 23, 20

(D) 20, 23, 25, 26, 36, 38, 39, 40

Answer: (B). percolateDown at indexes 4, 3, 2, 1 in turn (A is the top-down result).

My answer

Q30. Unit 09 Activity 02 (T252)

Convert the array 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 into a MaxHeap using the bottom-up method. The result is:

(A) 16, 14, 10, 8, 7, 3, 9, 1, 4, 2

(B) 16, 14, 10, 8, 7, 9, 3, 2, 4, 1

(C) 16, 14, 10, 9, 8, 7, 4, 3, 2, 1

(D) 16, 4, 10, 14, 7, 9, 3, 2, 8, 1

Answer: (B). percolateDown at indexes 5, 4, 3, 2, 1 (A is the top-down result; D is the array before the last step).

My answer

Q31. Assignment 3 (T252)

Build a max-heap from the array 14, 7, 26, 3, 19, 8, 2, 23, 11, 5 using the bottom-up heapify method. The final heap is:

(A) 26, 23, 14, 19, 7, 8, 2, 3, 11, 5

(B) 26, 23, 19, 14, 11, 8, 7, 5, 3, 2

(C) 26, 23, 14, 11, 19, 8, 2, 3, 7, 5

(D) 26, 19, 23, 11, 7, 8, 2, 3, 14, 5

Answer: (C). Index 5: no change; index 4: 3 and 23 swap; index 3: no change; index 2: 7 sinks below 23 and 11; index 1: 14 swaps with 26.

My answer

Q32. Quiz 4 Ver A (T251)

Using the bottom-up algorithm to build a max-heap, convert the sorted array 1, 4, 5, 7, 9, 12 into a max-heap. The result is:

(A) 12, 7, 9, 1, 5, 4

(B) 12, 9, 5, 7, 4, 1

(C) 12, 9, 7, 5, 4, 1

(D) 12, 9, 5, 4, 7, 1

Answer: (B). Index 3: 5 and 12 swap; index 2: 4 and 9 swap; index 1: 1 sinks below 12 and then 5 (A is the top-down result).

My answer

Q33. Final exam (T211)

Show the result of converting the array 65, 24, 63, 60, 27, 14, 11, 8, 21, 26 into a min-heap using the bottom-up approach.

(A) 8, 11, 14, 21, 26, 63, 24, 65, 27, 60

(B) 8, 21, 11, 24, 26, 14, 63, 60, 65, 27

(C) 8, 11, 14, 21, 24, 26, 27, 60, 63, 65

(D) 8, 21, 11, 26, 24, 14, 63, 60, 65, 27

Answer: (B). percolateDown at indexes 5 down to 1 (A is the top-down result).

My answer

Q34. Final exam (T181)

Create a min-heap from the array 9, 5, 3, 8, 7, 10, 11, 15, 6, 4, 2, 1 using the bottom-up approach (Floyd's algorithm). The final heap is:

(A) 1, 3, 2, 7, 4, 5, 11, 15, 9, 8, 6, 10

(B) 1, 2, 3, 6, 4, 9, 11, 15, 8, 5, 7, 10

(C) 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 15

(D) 1, 2, 3, 6, 4, 10, 11, 15, 8, 5, 7, 9

Answer: (B). percolateDown at indexes 6, 5, 4, 3, 2, 1 in turn (A is the top-down result).

My answer

Q35. Unit 09 Activity 02 (T252)

Heap sort with a max-heap is applied to the array 59, 36, 58, 21, 41, 97, 31, 16, 26, 53. What is the array after the build-heap phase (bottom-up)?

(A) 97, 59, 58, 53, 41, 36, 31, 26, 21, 16

(B) 97, 53, 59, 26, 41, 58, 31, 16, 21, 36

(C) 97, 53, 58, 26, 41, 59, 31, 16, 21, 36

(D) 97, 41, 59, 26, 53, 58, 31, 16, 21, 36

Answer: (B). This is the max-heap shown in the slides' heap sort example.

My answer

Q36. Unit 09 Activity 02 (T252)

Continue the heap sort (with no extra storage) of the previous question from the max-heap 97, 53, 59, 26, 41, 58, 31, 16, 21, 36. Each step swaps the root with the last element of the heap, reduces the heap size by 1, and percolates the new root down. What is the array after three such steps?

(A) 53, 41, 36, 26, 16, 21, 31, 58, 59, 97

(B) 58, 53, 36, 26, 41, 21, 31, 16, 59, 97

(C) 53, 36, 41, 26, 16, 21, 31, 58, 59, 97

(D) 41, 31, 36, 26, 16, 21, 53, 58, 59, 97

Answer: (A). Step 1: 59, 53, 58, 26, 41, 36, 31, 16, 21, 97; step 2: 58, 53, 36, 26, 41, 21, 31, 16, 59, 97; step 3: (A).

My answer

Q37. **[new]**

Which statement about heap sort as covered in the slides is correct?

(A) To sort an array in increasing order with no extra storage, a min-heap is used.

(B) To sort an array in increasing order with no extra storage, a max-heap is used, and the sort is $O(n \log n)$.

(C) Heap sort is $O(n)$ because building the heap bottom-up is $O(n)$.

(D) Heap sort is $O(n^2)$ because each of the n deletions may cost $O(n)$.

Answer: (B). Each deleted maximum is stored in the cell just freed at the end; n deletions of $O(\log n)$ each give $O(n \log n)$.

My answer

Q38.

Midterm (T231)

A priority queue is implemented using an unsorted array, where new entries are added at the end of the array, and entries with the highest priority are removed from the array. The time complexities of the enqueue and dequeue operations, respectively, are:

(A) $O(1)$, $O(1)$

(B) $O(1)$, $O(\log n)$

(C) $O(n)$, $O(1)$

(D) $O(1)$, $O(n)$

Answer: (D). Appending is constant; dequeue must scan all entries to find the highest priority.

My answer

Q39.

[new]

A minimum priority queue is implemented with a MinHeap of n keys. The worst-case complexities of Enqueue, Find Min and Dequeue Min are, respectively:

(A) $O(\log n)$, $O(1)$, $O(\log n)$

(B) $O(1)$, $O(n)$, $O(n)$

(C) $O(\log n)$, $O(\log n)$, $O(\log n)$

(D) $O(n)$, $O(1)$, $O(1)$

Answer: (A). Insert percolates up, the minimum is always at index 1, and deleteMin percolates down. (B), (D) and (C) are the unsorted list, sorted list and AVL tree rows of the slides' table.

My answer

Q40.

Midterm A (T222)

P is an initially empty priority queue that holds pairs (info, priority). A larger priority value means a higher priority, and items with equal priority leave in the order they arrived. Values are listed from the

front (first to leave) to the rear. What are the contents of P after the following operations, in the given order?

P.enqueue((3,12)), P.enqueue((8,15)), P.enqueue((4,13)), P.enqueue((0,11)), P.enqueue((5,14)), P.peek(), P.peek(), P.enqueue((1,19)), P.dequeue(), P.dequeue(), P.enqueue((6,17)), P.enqueue((2,18)), P.dequeue(), P.enqueue((7,11)), P.enqueue((10,18)), P.peek(), P.dequeue(), P.dequeue()

(A) ⟨(5,14), (4,13), (3,12), (0,11), (7,11)⟩

(B) ⟨(0,11), (7,11)⟩

(C) ⟨(1,19), (2,18), (10,18), (6,17), (8,15)⟩

(D) ⟨(10,18), (6,17)⟩

Answer: (A). The dequeues remove (1,19), (8,15), (2,18), (10,18), (6,17); peek removes nothing.

My answer

Part B: Programming

For Questions 1, 2 and 5–7, 9, use the following class (as in the slides, the heap is stored in `heap[1..heapSize]` and `heap[0]` is unused):

```
public class MinHeap {
    private int[] heap;     // heap[1..heapSize]
    private int heapSize;  // number of keys in the heap
    public MinHeap(int capacity) {
        heap = new int[capacity + 1];
        heapSize = 0;
    }
    private void percolateUp(int i)   { ... }  // as in Q1
    private void percolateDown(int i) { ... }  // as in Q2
}
```

Q41. [new]

Write the methods `public void insert(int key)` and `private void percolateUp(int i)` of class `MinHeap`. `insert` throws an `IllegalStateException` when the heap is full. Example: inserting 15 into 13, 18, 16, 24, 21, 19, 68, 65, 26, 32, 31 gives 13, 18, 15, 24, 21, 16, 68, 65, 26, 32, 31, 19.

Answer:

```
public void insert(int key) {
    if (heapSize == heap.length - 1)
        throw new IllegalStateException("Heap is full");
    heapSize++;
    heap[heapSize] = key;
    percolateUp(heapSize);
}

private void percolateUp(int i) {
    while (i > 1 && heap[i] < heap[i / 2]) {
        int temp = heap[i / 2];
        heap[i / 2] = heap[i];
        heap[i] = temp;
        i = i / 2;
    }
}
```

My answer

Q42. [new]

Write the methods `public int deleteMin()` and `private void percolateDown(int i)` of class

`MinHeap`. `deleteMin` removes and returns the minimum key and throws a `NoSuchElementException` when the heap is empty. Follow the slides: replace the root by the last key, delete the last node, and percolate down (move to the smaller child; use the right child only if it is strictly smaller than the left).

Answer:

```
public int deleteMin() {
    if (heapSize == 0)
        throw new NoSuchElementException("Heap is empty");
    int min = heap[1];
    heap[1] = heap[heapSize];
    heapSize--;
    if (heapSize > 0) percolateDown(1);
    return min;
}

private void percolateDown(int i) {
    int temp = heap[i];
    boolean done = false;
    while (!done && 2 * i <= heapSize) {      // node has a child
        int child = 2 * i;                      // left child
        if (child + 1 <= heapSize && heap[child + 1] < heap[child])
            child++;                            // right child is smaller
        if (heap[child] < temp) {
            heap[i] = heap[child];              // move child up
            i = child;
        } else done = true;
    }
    heap[i] = temp;
}
```

My answer

Q43. [new]

Write a static method `public static boolean isMinHeap(int[] a, int n)` that returns `true` if the keys `a[1..n]` (index 0 unused) satisfy the MinHeap property. Example: 13, 21, 16, 24, 31, 19, 68, 65, 26, 32 → `true`; 12, 20, 21, 6, 29, 19, 48, 60, 26, 30 → `false`.

Answer:

```
public static boolean isMinHeap(int[] a, int n) {
    for (int i = 2; i <= n; i++)
        if (a[i] < a[i / 2])     // child smaller than its parent
            return false;
    return true;
```

```
}
```

My answer

Q44. [new]

Class `MaxHeap` has fields `private int[] heap;` (keys in `heap[1..heapSize]`) and `private int heapSize;`. Write the constructor `public MaxHeap(int[] array)`, which copies `array` (0-based) into the heap and calls `buildHeapBottomUp()`, and write `buildHeapBottomUp()` and the max-heap version of `percolateDown(int i)`. Example: 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 becomes 16, 14, 10, 8, 7, 9, 3, 2, 4, 1.

Answer:

```
public MaxHeap(int[] array) {
    heapSize = array.length;
    heap = new int[heapSize + 1];
    for (int i = 0; i < array.length; i++)
        heap[i + 1] = array[i];
    buildHeapBottomUp();
}

private void buildHeapBottomUp() {
    for (int i = heapSize / 2; i >= 1; i--)   // last non-leaf down to root
        percolateDown(i);
}

private void percolateDown(int i) {
    int temp = heap[i];
    boolean done = false;
    while (!done && 2 * i <= heapSize) {
        int child = 2 * i;
        if (child + 1 <= heapSize && heap[child + 1] > heap[child])
            child++;                          // larger child
        if (heap[child] > temp) {
            heap[i] = heap[child];
            i = child;
        } else done = true;
    }
    heap[i] = temp;
}
```

My answer

Q45. [new]

Suppose the array `heap[1..heapSize]` of a `MinHeap` has been filled with arbitrary keys. Write `public void buildHeapTopDown()` that converts it into a min-heap using the slides' top-down heapify (use `percolateUp` from Q1). Example: 70, 29, 68, 65, 32, 19, 16, 13, 26, 31 becomes 13, 16, 19, 26, 31, 68, 29, 70, 32, 65. What is its complexity?

Answer:

```
// Complexity: O(n log n)
public void buildHeapTopDown() {
    for (int i = 2; i <= heapSize; i++)   // second element to the last
        percolateUp(i);
}
```

My answer

Q46. [new]

Write `public boolean deleteKey(int key)` in class `MinHeap` that deletes an arbitrary key: find it, copy the key of the last node into its position, delete the last node, and restore the heap property. Return `false` if the key is not in the heap. Example: deleting 25 from 10, 20, 25, 28, 20, 30, 40, 30, 90, 70, 80, 60, 32, 50, 40, 35 gives 10, 20, 30, 28, 20, 32, 40, 30, 90, 70, 80, 60, 35, 50, 40.

Answer:

```
public boolean deleteKey(int key) {
    int pos = -1;
    for (int i = 1; i <= heapSize; i++)
        if (heap[i] == key) { pos = i; break; }
    if (pos == -1) return false;
```

```
    heap[pos] = heap[heapSize];   // copy last node's key
    heapSize--;                   // delete last node
    if (pos <= heapSize) {
        percolateDown(pos);       // the copied key may be too large
        percolateUp(pos);         // or smaller than its new parent
    }
    return true;
}
```

My answer

Q47. [new]

Write `public void changeKey(int i, int newKey)` in class `MinHeap` that changes the key at index `i` to `newKey` and percolates up, percolates down, or does nothing, as needed. Throw an `IndexOutOfBoundsException` for an invalid index. Example: in 18, 21, 19, 24, 31, 23, 68, 65, 26, 32, changing index 2 (key 21) to 30 gives 18, 24, 19, 26, 31, 23, 68, 65, 30, 32.

Answer:

```
public void changeKey(int i, int newKey) {
    if (i < 1 || i > heapSize)
        throw new IndexOutOfBoundsException();
    int old = heap[i];
    heap[i] = newKey;
    if (newKey < old)
        percolateUp(i);         // higher priority
    else if (newKey > old)
        percolateDown(i);       // lower priority
}                               // equal: no percolation
```

My answer

Q48. [new]

Write a static method `public static void heapSort(int[] a, int n)` that sorts `a[1..n]` (index 0 unused) in increasing order with no extra storage, as in the slides: build a max-heap with `percolateDown`, then repeatedly swap the root with the last element of the heap, reduce the heap size by 1, and percolate the new root down. Write the helper `percolateDown(int[] a, int i, int size)` too. Example: 59, 36, 58, 21, 41, 97, 31, 16, 26, 53 → 16, 21, 26, 31, 36, 41, 53, 58, 59, 97.

Answer:

```
public static void heapSort(int[] a, int n) {
    for (int i = n / 2; i >= 1; i--)        // build max-heap
        percolateDown(a, i, n);
    for (int size = n; size > 1; size--) {
        int temp = a[1];                    // move max to the end
        a[1] = a[size];
        a[size] = temp;
        percolateDown(a, 1, size - 1);      // fix the smaller heap
    }
}

private static void percolateDown(int[] a, int i, int size) {
    int temp = a[i];
    while (2 * i <= size) {
        int child = 2 * i;
        if (child + 1 <= size && a[child + 1] > a[child])
            child++;
        if (a[child] > temp) {
            a[i] = a[child];
            i = child;
        } else break;
    }
    a[i] = temp;
}
```

My answer

Q49. [new]

Write `public int findMax()` in class `MinHeap` that returns the largest key, examining only the leaves (indexes $\lfloor heapSize/2 \rfloor + 1$ to `heapSize`). Throw a `NoSuchElementException` if the heap is empty. Example: 13, 21, 16, 24, 31, 19, 68, 65, 26, 32 → 68.

Answer:

```
public int findMax() {
    if (heapSize == 0)
        throw new NoSuchElementException("Heap is empty");
    int max = heap[heapSize];
    for (int i = heapSize / 2 + 1; i <= heapSize; i++)  // leaves only
        if (heap[i] > max) max = heap[i];
    return max;
}
```

My answer

Q50. [new]

A priority queue of jobs uses a max-heap stored in an array `Job[] heap` (index 0 unused); a larger `priority` means a higher priority.

```
class Job {
    String name; int priority;
    Job(String name, int priority) { this.name = name; this.priority = priority; }
}
public class JobQueue {
    private Job[] heap;
    private int heapSize;
    public JobQueue(int capacity) { heap = new Job[capacity + 1]; heapSize = 0; }
    public boolean isEmpty() { return heapSize == 0; }
}
```

Write `public void enqueue(Job job)` (throw `IllegalStateException` if full) and `public Job dequeue()` (remove and return the job with the highest priority; throw `NoSuchElementException` if empty). Example: enqueue a(12), b(15), c(13), d(19), e(11); dequeuing all gives d, b, c, a, e.

Answer:

```
public void enqueue(Job job) {
    if (heapSize == heap.length - 1)
        throw new IllegalStateException("Queue is full");
    heapSize++;
    int i = heapSize;
    while (i > 1 && job.priority > heap[i / 2].priority) {
        heap[i] = heap[i / 2];          // move parent down
        i = i / 2;
    }
    heap[i] = job;                      // percolate up done
}

public Job dequeue() {
    if (heapSize == 0)
        throw new NoSuchElementException("Queue is empty");
    Job top = heap[1];
    Job temp = heap[heapSize];          // last job
```

```
    heap[heapSize] = null;
    heapSize--;
    if (heapSize > 0) {
        int i = 1;                      // percolate temp down from root
        while (2 * i <= heapSize) {
            int child = 2 * i;
            if (child + 1 <= heapSize
                    && heap[child + 1].priority > heap[child].priority)
                child++;
            if (heap[child].priority > temp.priority) {
                heap[i] = heap[child];
                i = child;
            } else break;
        }
        heap[i] = temp;
    }
    return top;
}
```

My answer